Computer Vision:

Week-11 (06 April--10 April) Theory Lectures

Dr. Irfan Ali

Assistant Professor (AI, Multimedia Gaming & Cybersecurity

& Program Coordinator of Multimedia & Gaming Deptt

Review

- ANN & DNN Networks
- Numerical
- Data augmentation, loss, optimizers, Transfer learning and fine-tuning.

Data Augmentation

Augmentation *artificially expands our dataset by applying transformations to the input pixels on the fly:*

Geometric: *Rotations, horizontal flips, scaling, and cropping.*

Photometric*: Adjusting brightness, contrast, and adding Gaussian noise.*

Why it works: *It forces the model to learn Invariance. The model learns that a "Any image such as dog picture" is still a "dog" even if it's upside down or in low light.*

Loss Functions

The Loss Function, L, quantifies the "error" of the model.

- *For Classification, we typically use Cross-Entropy Loss.*

$$L = -\sum_{i} y_i \log(\hat{y}_i)$$

- *where y is the ground truth and* $(\hat{y}_i)$ *is the prediction.*
- *For Regression (like bounding box coordinates), use Mean Squared Error (MSE).*

Optimizers

In a Convolutional Neural Network,

navigating a landscape with millions of dimensions. Choosing the right optimizer is the difference between finding the global minimum and getting stuck in a local valley or a plateau

Optimizers

When a model makes a mistake (calculated by the Loss Function), it needs to know exactly how to change its internal weights to do better next time.

***The optimizer** is the mathematical algorithm that determines the direction and the speed (learning rate) of those changes.*

Optimizers

If the Loss Function is a mountain range and the bottom of the valley is "perfection," the Optimizer is the pathfinder leading the model down the mountain.

Types of Optimizers: *generally categorize them based on how they handle the "speed" of learning:*

Optimizers

1. *Constant Learning Rate Optimizers: These apply the same logic to every update, regardless of how the training is going.*

-SGD (Stochastic Gradient Descent): The most basic form. It takes a small step toward the "downhill" direction for every batch of images. It's reliable but can be very slow and sometimes gets stuck in "puddles" (local minima) on the way down.

Optimizers

2. Momentum-Based Optimizers: These add a "physics" element to the movement.

-SGD with Momentum: *Imagine a heavy ball rolling down a hill. Because of its "momentum," it gains speed as it goes and can roll right over small bumps or flat spots that might stop regular SGD. It helps the model stay on track and reach the bottom faster.*

Optimizers

3. Adaptive Learning Rate OptimizersThese are the "smart" optimizers. They change the speed for every single parameter in the network based on its history.

Adagrad: It gives a larger learning rate to features that appear rarely and a smaller one to features that appear often.

Optimizers

RMSProp: An improvement on Adagrad that prevents the learning rate from getting too small too early, keeping the training alive.

Adam (Adaptive Moment Estimation): Currently the "King" of optimizers. It combines the best of Momentum and RMSProp. It tracks the moving average of the gradient and its square. It is the default choice for almost every second-year project because it requires very little manual tuning to work well.

Optimizers

Type	Optimizer	Characterization	Best For...
Constant LR	**SGD** (Stochastic Gradient Descent)	Updates weights using one sample/batch at a time. High variance in updates.	Simple models or very large datasets.
Momentum	**SGD + Momentum**	Adds a fraction of the previous update to the current one. Helps "push" through local minima.	High-performance vision models where stability is key.
Adaptive LR	**AdaGrad / RMSProp**	Scales the learning rate based on how frequently a feature appears.	Sparse data or specific NLP/Vision overlaps.
The Hybrid	**Adam** (Adaptive Moment Estimation)	Combines **Momentum** and **RMSProp**. It tracks both the average and the variance of gradients.	**Default choice** for most CNN architectures today.

Transfer Learning

Transfer Learning *is the process of taking a model already trained on a massive dataset (like ImageNet, which has 1.4 million images) and repurposing its "knowledge" for a specific task.*

The Intuition: *The first few layers of a CNN learn generic features (edges, blobs, colors). These features are universal. Whether you are detecting skin cancer or identifying a Tesla, an "edge" is still an "edge."*

The Strategy: Freezing vs. Fine-Tuning

Load Pre-trained Weights: We take a model like ResNet-50 or EfficientNet trained on ImageNet.

Replace the "Head": The original model was designed to classify 1,000 classes. We chop off the last layer and replace it with a new layer matching our number of classes (e.g., "Malignant" vs. "Benign").

The Strategy: Freezing vs. Fine-Tuning

*Fr**eezing (Feature Extraction):** "freeze" the weights of the early layers so they don't change. only train the new head.*

***Fine-Tuning:** Once the head is stable, we unfreeze the later layers and retrain with a very low learning rate. This allows the model to subtly adjust its high-level filters to the specific nuances of our data.*

The Strategy: Freezing vs. Fine-Tuning

Dataset Size	Data Similarity	Strategy
Small	High	**Feature Extraction:** Freeze all layers, train only the new classifier.
Small	Low	**Tricky:** Freeze early layers, but you might need to retrain from a middle point.
Large	High	**Fine-Tuning:** Unfreeze the last few layers to optimize performance.
Large	Low	**Full Training:** You can afford to unfreeze the whole network or train from scratch.

The Strategy: Freezing vs. Fine-Tuning

Model Name	*Year*	*The "Big Idea"*	*Simple Description*	*Best Used For...*
VGG-16 / 19	2014	**Uniformity**	It uses very small 3 by 3 filters stacked repeatedly. It's like a very deep, straight hallway.	Learning how basic convolutions work.
ResNet	2015	**Skip Connections**	It introduced "Residual" blocks that let information skip layers. This prevented the "Vanishing Gradient" problem.	The standard "Go-to" for almost any project.
Inception (GoogLeNet)	2014	**Parallelism**	Instead of picking one filter size, it uses many 1 by 1 or 3 by 3 at once in the same layer.	Efficiency and complex pattern recognition.
MobileNet	2017	**Lightweight**	Uses "Depthwise" convolutions to reduce math operations. It is designed to run on a smartphone, not a supercomputer.	Apps, drones, and mobile devices.

The Strategy: Freezing vs. Fine-

Model Family	*Key Innovation*	*Why it's "Advanced"*	*Use Case*
ViT (Vision Transformer)	**Self-Attention**	It treats an image like a "sentence" made of patches. It understands how a top-left pixel relates to a bottom-right pixel immediately.	High-accuracy research and large-scale classification.
DINOv2	**Self-Supervised**	Trained by Meta on 142M images *without labels*. It has a "god-like" understanding of object depth and boundaries.	The best "Backbone" to freeze and use for any custom task.
YOLO26	**Real-Time Speed**	The latest in the YOLO family. It is incredibly fast and can run on your smartphone with near-	Real-time object detection (Robotics, self-driving).

The Strategy: Freezing vs. Fine-Tuning

SAM 2 (Segment Anything)	**Zero-Shot Prompting**	You can "prompt" it with a click or a word to cut out any object perfectly, even if it has never seen that object before.	Medical imaging or video editing.
CLIP / Gemini	**Multimodal**	These models understand both **Images and Text** together. They can "reason" about what they see.	Searching images using natural language (e.g., "Find the red car near the tree").

N1: A model process grayscale images (0-255) scaled 0-1 initial weight 0.5 gradient 0.2, and learning rate 0.1 , update the weight using gradient descent

Variables and Parameters

Based on your input, here are the values we are working with:

- **Initial Weight (w):** 0.5
- **Gradient (g):** 0.2
- **Learning Rate (eta):** 0.1

N1:

The Calculation

The weight update formula is defined as:

$$w_{new} = w_{old} - (\eta \times g)$$

N1:

Calculate the step size: Multiply the learning rate by the gradient.

$0.1 \times 0.2 = 0.02$

Update the weight: Subtract this step from the initial weight.

$0.5 - 0.02 = 0.48$

N1-Practice: **Finding the Gradient**

A neural network is processing scaled image data. You are given the following parameters after one training step:

- Initial Weight w_{old} = 0.80
- Updated Weight 0.74

Learning Rate : 0.2

Using the gradient descent update rule, calculate the gradient (g) that was used to arrive at the new weight.

N2: Given a single weight in a model, calculate the updated weight after one step of the Adam optimizer using these values:

Initial Weight $(w_{old}) : 0.80$

$$Gradient(g) : 0.3$$

$$Learning\ Rate(\eta) : 0.1$$

N2:

$First\ Moment\ Decay\ (\beta_1): 0.9\ (standard\ default)$

$Second\ Moment\ Decay(\beta_2): 0.999\ (standard\ default)$

$Epsilon(\epsilon \quad , prevent\ division\ by\ zero)$

Initial Moments ($\mathbf{m}_0$ and v_0): 0

Step-1 Update the First Moment (Mean)

$$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t$$

$$m_1 = 0.9 \cdot 0 + (1-0.9) \cdot 0.3 = \mathbf{0.03}$$

Step-2 Update the Second Moment (Uncentered Variance)

$$v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2$$

$$v_1 = 0.999 \cdot 0 + (1-0.999) \cdot 0.3^2 = 0.001 \cdot 0.09 = \mathbf{0.00009}$$

Step-3 Apply Bias Correction

$$\hat{m}_t = \frac{m_t}{1-\beta_1^t} = \frac{0.03}{1-0.9^1} = \frac{0.03}{0.1} = \mathbf{0.3}$$

$$\hat{v}_t = \frac{v_t}{1-\beta_2^t} = \frac{0.00009}{1-0.999^1} = \frac{0.00009}{0.001} = \mathbf{0.09}$$

Step-4 Update the weight

$$w_1 = w_0 - \eta \cdot \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

$$w_1 = 0.80 - 0.1 \cdot \frac{0.3}{\sqrt{0.09} + 10^{-8}}$$

$$w_1 = 0.80 - 0.1 \cdot \frac{0.3}{0.3}$$

$$w_1 = 0.80 - 0.1 \cdot 1 = \mathbf{0.70}$$

N2-Practice: Given a single weight in a model, calculate the updated weight after one step of the Adam optimizer using these values:

Initial Weight $(w_0): 1.20$

Gradient $(\mathbf{g}): 0.15$

Learning Rate $(\eta): 0.05$

First Moment Decay $(\beta_1): 0.9$

Second Moment Decay $(\beta_2): 0.99$

Epsilon $(\epsilon$

Initial Moments $(m_0,\ \mathrm{v}_0): 0$